

```
pg_restore: finished item 3220 FK CONSTRAINT j_posts_tags j_posts_tags_
post_pk_fkey
pg_restore: finished main parallel loop
```

Thanks to the verbose flag, it is clear how `pg_restore` has executed a parallel restoration of the data in the database. Messages such as `launching item` and `finished item` indicate when and on what object a parallel worker has been involved.

Backup automation

By combining `pg_dump` and `pg_dumpall`, it is quite easy to create automated backups, for example, to run every night or every day when the database system is not heavily used. Depending on the operating system you use, it is possible to schedule such backups and have them executed and rotated automatically.

If you're using Unix, for example, it is possible to schedule `pg_dump` via `cron`(1), as follows:

```
$ crontab -e
```

After doing this, you would add the following line:

```
30 23 * * * pg_dump -Fc -f /backup/forumdb.backup -U forum forumdb
```

This initiates a full backup in a custom format every day at 23:30. However, the preceding approach has a few drawbacks, such as managing already existing backups, dealing with newly added databases that require another line to be added to the crontab, and so on.

Thanks to the flexibility of PostgreSQL and its catalog, it is simple enough to develop a wrapper script that can handle backing up all the databases with ease. As a starting point, the following script performs a full backup of every database except for `template0`:

```
#!/bin/sh

BACKUP_ROOT=/backup

for database in $( psql -U postgres -A -t -c "SELECT datname FROM pg_
database WHERE datname <> 'template0'" postgres )
do
    backup_dir=$BACKUP_ROOT/$database/${date +%Y-%m-%d'}
    if [ -d $backup_dir ]; then
        echo "Skipping backup $database, already done today!"
```

```
        continue
    fi

    mkdir -p $backup_dir
    pg_dump -U postgres -Fd -f $backup_dir $database
    echo "Backup $database into $backup_dir done!"
done
```

The idea is quite simple: the system queries the PostgreSQL catalog, `pg_database`, for every database that the cluster serves, and for every database, it searches for a dedicated directory, named after the database that contains a directory named after the current date. If the directory exists, the backup has already been done, so there is nothing to do but continue to the next database. Otherwise, the backup can be performed. Therefore, the system will back up the `forumdb` database to the `/backup/forumdb/2023-07-19` directory one day, `/backup/forumdb/2023-07-20` the next day, and so on. Due to this, it is simple to add the preceding script to your crontab and forget about adding new lines for new databases, as well as removing lines that correspond to deleted databases:

```
30 23 * * * my_backup_script.sh
```

Of course, the preceding script does not represent a complex backup system but, rather, a starting point if you need a quick and flexible solution to perform an automated logical backup, with the tools your PostgreSQL cluster and operating system offer. As already stated, many operating systems have already taken backing up a PostgreSQL cluster into account and offer already crafted scripts to help you solve this problem. A very good example of this kind of script is the `502.pgsql` script, which is shipped with the FreeBSD package of PostgreSQL.

The COPY command

The `COPY` command is not a backup facility by design, but it is very efficient in bulk loading of data and, therefore, in backup restores, as already mentioned. However, this command can be used on its own to load, in a bidirectional way, data: thanks to `COPY` you can extract data from a table (i.e., do a dump) or load data into a table (i.e., do a restore). Moreover, `COPY` can interact with external programs; that is, it can send (or receive) data directly from another process.

`COPY` has two main operating modes:

- `COPY TO` pulls data out of a table and sends it to a file on the filesystem or to another external application or process.

- `COPY FROM` loads data from a file on the filesystem or an external application and inserts it into a specified table.



When dealing with external files or programs, the `COPY` command requires superuser privileges (or at least for the user to belong to the `pg_write_server_files` group). For this reason, examples shown in this section will be run as the `postgres` superuser.

As a simple example, imagine we need to extract all the data from the `categories` table; this is a `COPY TO` kind of command, as specified below:

```
forumdb=# COPY forum.categories TO '/tmp/categories.backup.txt';
COPY 5
```

In the above example, the content of the table is written to the local file, `/tmp/categories.backup.txt`, and the file content is made only by the tuples without any particular SQL instruction:

```
$ cat /tmp/categories.backup.txt
5      Software engineering      Software engineering discussions
1      Database                  Database related discussions
2      Unix                      Unix and Linux discussions
3      Programming Languages     All about programming languages
4      A.I                      Machine Learning discussions
```

The `COPY` command supports a wide range of options that allow the user to define a field delimiter, the presence of the table header (i.e., column names), a quoting character, and so on. This makes it very simple to build your own **comma separated values (CSV)** set of data:

```
forumdb=# COPY forum.categories TO '/tmp/categories.csv'
WITH ( HEADER on, DELIMITER ';' );
COPY 5

$ cat /tmp/categories.csv
pk;title;description
5;Software engineering;Software engineering discussions
1;Database;Database related discussions
2;Unix;Unix and Linux discussions
3;Programming Languages;All about programming languages
```

4;A.I;Machine Learning discussions

As you can see from the above example, all the fields are now separated by a semicolon, and the first row in the file is the table column list. It is worth noting that COPY already comes with a defined CSV format, which can be specified with the option `FORMAT csv`:

```
forumdb=# COPY forum.categories TO '/tmp/categories.csv'
WITH ( FORMAT csv );
COPY 5
```

Having a CSV file containing the data means that `COPY FROM` can load the data into a table. In other words, COPY is not only an efficient bulk loader; it is also a useful tool to load data into the database from external resources like spreadsheets.

Imagine we need to load the data contained in the `categories.csv` file into another table, named `categories_reloaded`:

```
forumdb=# CREATE TABLE forum.categories_reloaded( LIKE forum.categories );
CREATE TABLE
forumdb=# COPY forum.categories_reloaded
          FROM '/tmp/categories.csv'
          WITH (FORMAT csv);
COPY 5
```

It is also possible to specify a `WHERE` clause, in order to filter what is going to be loaded; as an example, imagine we want to load only odd rows:

```
forumdb=# COPY forum.categories_reloaded
          FROM '/tmp/categories.csv'
          WITH (FORMAT csv)
          WHERE pk % 2 = 1;
```

The `COPY TO` does not allow for a `WHERE` clause, but it is possible to *copy from a query*, which does the trick of filtering tuples:

```
forumdb=# COPY
( SELECT * FROM forum.categories
  WHERE pk % 2 = 1 )
TO '/tmp/categories.odd.csv'
WITH (FORMAT csv);
```

It is also possible to pull tuples from an external application, as the following trivial example demonstrates:

```
forumdb=# COPY forum.categories_reloaded
FROM PROGRAM $CODE$
/bin/bash -c 'for i in {1..10}; do echo "$i,Title$i,A generated row";
done' $CODE$
WITH (FORMAT csv);
```

In the above example, a shell process is launched; such a process generates 10 tuples in the CSV format. The COPY command pulls the tuples from the standard output of the command and inserts them into the table. This acts as an operating system pipe between PostgreSQL tables and external processes.

Similarly, it is also possible to send data to an external program, as the following simple example does:

```
forumdb=# COPY forum.categories
          TO PROGRAM $CODE$ awk '{print $2;}' > /tmp/titles.txt $CODE$;
COPY 5

$ cat /tmp/titles.txt
Software
Database
Unix
Programming
A.I
```

In the above example, the whole data contained in the table is sent to `awk(1)`, which extracts only the second column (i.e., the title column) and redirects its own output to a file. The end result is a kind of pseudo filtering of the table content.



It is worth noting that COPY supports single-column lists, which means you can already specify which columns you are extracting or inserting into.

When using COPY to deal with external files or programs, the user running the command must be a superuser or must belong to the `pg_write_server_files` group.

In other words, COPY is not usable by unprivileged users. To deal with this, psql provides its own COPY replacement command, named `\copy`, that streams the content (in either direction) with regard to files accessible to the psql client. This way, the user is able to exploit COPY without needing any particular server privilege like `pg_write_server`.

Therefore, to extract data from a query and place it into a file, an unprivileged user should do something like:

```
forumdb=> \COPY
( SELECT * FROM forum.categories
  WHERE pk % 2 = 1 )
TO '/tmp/categories.odd.csv'
WITH (FORMAT csv);
COPY 3
```

Note the usage of `\copy` instead of COPY.

Thanks to the availability of COPY, and its psql wrapper `\copy`, the user can easily bulk-load data and extract it from the database.

Now that we've explored logical backups, let's move on to physical ones.

Exploring physical backups

A physical backup is a low-level backup that's taken during the normal operations of the database cluster. Here, low-level means that the backup is somehow performed “externally” to the backup cluster – that is, at the filesystem level.

As you already know from *Chapter 10, Users, Roles, and Database Security*, the database cluster requires both the data files contained in PGDATA/base and the WALs contained in PGDATA/wal, as well as a few other files, to make the cluster work properly. The main concept, however, is that the data files and the WALs can make the cluster self-healing and recover from a crash. Hence, a physical backup performs a copy of all the cluster files and then, when the restore is required, it simulates a database crash and makes the cluster self-heal with the WALs in place.

The reason why physical backups are important is that they allow us to effectively clone a cluster, starting from the files it is made of. This means, on one hand, that you cannot restore a physically backed-up cluster on a different PostgreSQL version and, on the other hand, that you need essentially no interaction at all with the cluster during the backup phase.

The last point is particularly important: the physical backup can be taken pretty much at every moment without impacting the database with a huge transaction, which occurs in logical backups, and without interfering with the ongoing database activities, such as client connections and queries. It is true that the storage system – in particular, the filesystem – will be put under pressure during this kind of backup, but to the cluster, the backup is almost transparent.

It is fair to say that the cluster must be informed that the backup has started, allowing it to clearly mark that a backup is in progress inside the WALs, but apart from this “simple” action, the backup is totally outside the scope of the database cluster.

Moreover, physical backups allow you to choose the best tool that fits the low-level file copy. You are free to use any filesystem-specific command, such as `cp(1)`, `rsync(1)`, `tar(1)`, and so on; you can do the backup via a network by using any file-copying mechanism provided by your operating system, and you can even develop your own tool. There are also a lot of backup solutions for PostgreSQL, including the authors’ favorite, `pgBackRest`, so you are free to tailor your backup strategy to the tools that best fit your environment and requirements.

In the following subsections, you will learn how to perform a physical backup by means of a tool shipped with PostgreSQL – `pg_basebackup`. This tool has been developed as the primary tool to clone a cluster, for example, as the starting point of a replicated system (replication will be shown in later chapters).

Please consider that, in any case, what `pg_basebackup` does is perform a set of steps that can be performed manually by any system administrator, so the tool is a convenient and well-tested way of doing a physical backup.

Performing a manual physical backup

The `pg_basebackup` tool performs either a local or remote database cluster clone operation that can be used as a backup. In order to work properly, the cluster that needs to be cloned must be set up accordingly. Since `pg_basebackup` “asks” PostgreSQL to provide the WALs, it is important that the target cluster has at least two WAL Sender processes active (WAL Sender processes are responsible for serving WALs over a client connection).

Therefore, the first step to perform on the database you want to back up is to check that the `max_wal_senders` configuration parameter (in the `postgresql.conf` file) has a value of 2 or greater:

```
max_wal_senders = 2
```

Another important setting is to allow `pg_basebackup` to perform a connection to the cluster: the tool will connect not as an ordinary client but as a “replication” client, and therefore, the `pg_hba.conf` file must allow a rule that allows an administrative user to connect to the “replication” special database. Something similar to the following should work for a local backup:

```
host    replication    postgres  127.0.0.1/32    trust
```

Here, the user `postgres` is allowed to connect from the very same host to the special replication database without providing any authentication credentials.



WARNING: a replication connection can copy every piece of data from the database and, therefore, must be protected as much as possible. In production environments, always limit incoming hosts for a replication connection, and also set up strong credentials to validate the connection!

Let’s assume we want to perform the physical backup to store the result – that is, the backup itself – in the `/backup/data` directory. In order to do the backup, the target directory must exist, and if the database has tablespaces, every single directory for a tablespace must be remapped to another directory. The latter is required because we make a backup on the very same host, so PostgreSQL prevents directory clashes.

A backup also needs a label – that is, a mnemonic description of the aim of the backup, which is for human-readability purposes only.



In the Docker image of this chapter, the directories required to perform the backup and the tablespace remapping are already configured.

The following command will perform the backup:

```
$ pg_basebackup -D /backup/data -l 'My Physical Backup' -v -h localhost -p
5432 -U postgres -T /data/tablespaces/ts_b=/backup/tablespaces/ts_b -T /
data/tablespaces/ts_a=/backup/tablespaces/ts_a -T /data/tablespaces/ts_c=/
backup/tablespaces/ts_c
pg_basebackup: initiating base backup, waiting for checkpoint to complete
pg_basebackup: checkpoint completed
pg_basebackup: write-ahead log start point: 0/2000028 on timeline 1
pg_basebackup: starting background WAL receiver
```

```
pg_basebackup: created temporary replication slot "pg_basebackup_117"  
pg_basebackup: write-ahead log end point: 0/2000100  
pg_basebackup: waiting for background process to finish streaming ...  
pg_basebackup: syncing data to disk ...  
pg_basebackup: renaming backup_manifest.tmp to backup_manifest  
pg_basebackup: base backup completed
```

The `-D` flag specifies the directory that you want the backup to be stored in, which in this example is `/backup/data`. The `-l` optional flag allows you to provide a textual label to your backup, which can be used to inspect the backup to get some extra information about it. The `-v` flag enables verbose mode, which produces rich output about what the command performs at every step. The repeated `-T` flag tells PostgreSQL how to remap every single directory that is used as a tablespace: the directory on the left of the equal sign is the existing directory from which the backup is taken, and the directory on the right is the remapped path.

The other arguments are typical PostgreSQL `libpq` client flags that specify how to connect to the database so that it can be cloned – in this case, by means of the user `postgres` on `localhost` at port `5432`.



`pg_basebackup` supports several other options that can be used to, for instance, limit network bandwidth usage, show the progress of ongoing backups, and much more. Please refer to the command documentation and online help for further details.

If you inspect the directory where the backup has been stored, you will see that it is effectively a clone of the `PGDATA` directory of the server you took the backup from, including its configuration files.

pg_verifybackup

Since PostgreSQL 13, a tool named `pg_verifybackup` can be used to verify the integrity of a backup done via `pg_basebackup`. At a glance, it works as follows:

```
$ pg_verifybackup /backup/data/  
backup successfully verified
```

Specifying the directory that contains the backup, the tool is able to perform a check and report for any corruption. The tool performs four main steps:

1. It evaluates the backup manifest to check if it is readable and contains valid backup information.
2. It scans the backup content to search for missing or modified data files (some configuration files are skipped in this step because the user could have changed them).
3. It compares all the data file checksums with the manifest values to ensure the files have not been corrupted.
4. By exploiting another utility, `pg_waldump`, it verifies that the WAL records that are needed to restore the backup are in place and readable.

Thanks to `pg_verifybackup`, you can be sure that your backup has not been damaged by a filesystem problem, a disk failure, or something else, and therefore, you can resume from such a backup.

Starting the cloned cluster

`pg_basebackup` does a complete clone of the target cluster, including the configuration files. This means that the configuration of the cluster has not been “adapted” to where the clone is, including the data directory and the listening options (for example, the TCP/IP port). Therefore, you must be careful when starting the cloned cluster, since it could clash with the original one, especially if the backup is performed locally (on the same machine).

Here, you have the option of editing the configuration before attempting to start the backup cluster, changing the main settings on the command line, or moving the backup to a remote host.

If you want to start the cloned cluster, assuming it has been kept local, as in the previous section, you can, for example, restart it with the following command-line settings:

```
$ pg_ctl -D /backup/data/ -o '-p 5433' start
waiting for server to start....
LOG:  database system was interrupted; last known up at 2023-07-19
16:43:39 UTC
LOG:  redo starts at 0/2000028
LOG:  consistent recovery state reached at 0/2000138
```